

THESIS PROPOSAL

Windowed Encoding and Phase-Aware Deep Reinforcement Learning for Deterministic Time-Dependent TSP



STUDENT: Anuradha Dissanayake, 1747776

SUPERVISOR: Tim Dornedde

Date: 22/08/2025

1. Motivation

The classical Traveling Salesman Problem (TSP) is a well-known problem in many fields such as computer science, operations research, transportation and the automation industry. The main goal of the traditional TSP is to find the shortest route that visits all customers at different locations exactly once. However, it assumes that the cost between any two cities is fixed and does not change over time. In reality, travel time between two locations can vary depending on the departure time due to factors like traffic congestion, rush hours, maintenance and others. The Deterministic Time-Dependent Traveling Salesman Problem (TDTSP) addresses this by allowing the cost of a move to depend on its departure time.

Conventional methods handle time dependence using heuristics such as meta-heuristics and nearest-neighbor variants. These methods can find good solutions, but they often require manual adjustments or continuous re-optimization, specially when start times change. In contrast, deep reinforcement learning (DRL) approaches this differently. Instead of repeatedly solving the problem from scratch, DRL trains a model offline and then uses a forward pass at runtime to make decisions almost instantly.

Among DRL methods, attention-based encoder-decoder policies have become strong baselines for routing problems. In particular, Zhang et al. propose two models called M1 and M2. Both models are suitable for TDTSP, but the M1 model is better suited for fixed customer bases, while M2 works for both fixed and dynamic customer bases. For deterministic TDTSP, M1 is attractive because of its speed, performance and simplicity. However, three practical gaps have been identified:

1. Feeding the full days' time series of edge costs to M1's encoder can be memory or compute intensive and unnecessary for a given start time.
2. The decoder in M1 is stateless across steps (beyond the visited mask and last node feature). Consistent with FIFO and the goal of forgetting irrelevant history, we do not add a recurrent memory (GRU).
3. The baseline decoder learns preferences from data but does not explicitly incorporate a cost-aware bias.

The primary motivation is to find solutions to these gaps. Most papers use the REINFORCE algorithm with a rollout baseline, but it is also necessary to try this method alongside the actor-critic method to compare performance.

2. State of the art

2.1 Problem scope

This work mainly focuses on the deterministic Time-Dependent Traveling Salesman Problem (TDTSP) with a fixed customer base. Decisions are made sequentially (the next node is chosen after each arrival) and we assume no waiting times at nodes.

2.2 Attention/Transformer policies for routing

Early neural network models for routing problems (such as pointer networks) used Recurrent Neural Networks (RNNs). However, RNNs can suffer from vanishing or exploding gradients when dealing with long sequences. To address this issue, researchers proposed pure attention-based models, specifically Transformer architectures. One of the most influential works in this area is Kool et al. (2019), who introduced an attention-based encoder-decoder model trained with reinforcement learning. The baseline paper by Zhang et al., which inspired this thesis, builds upon this approach with a few modifications.

2.3 Zhang et al. baseline (M1/M2)

The baseline paper focuses on both static and dynamic customer pools in the TDTSP and also considers the dynamic pickup and delivery problem. However we restrict our study to the static TDTSP and remove some parts of the model accordingly.

- **M1 Model (Encode Once, Decode Online)**

The static encoder processes static node information and estimated traffic patterns in the TDTSP context to obtain node and graph embeddings. The decoder then chooses the next node step by step using the current travel time vector and a visited mask, without re-encoding. This approach is fast and effective when the customer set is fixed.

- **M2 Model (Re-encode Every Step)**

This model uses pre-trained Node2Vec (which focuses mainly on node information) and Traffic2Vec models (capturing current traffic conditions and historical traffic patterns). Then it fine-tunes and runs the decoder to generate the next visited state. M2 is better suited for dynamic customer pools but is computationally heavier.

2.4. Training algorithms (REINFORCE vs Actor-Critic)

- **REINFORCE (policy gradient) with a baseline**

Run a full tour to obtain return R (negative total tour time) and update the policy using:

$$\nabla_{\theta} E[R] \approx \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (R - b)$$

In routing, it is common to use stepwise rewards and Monte-Carlo returns G_t with advantage $A_t = G_t - b_t$. Additional techniques such as advantage normalization, gradient clipping and a small entropy bonus are also used.

- **Actor-Critic methods**

These combine a policy network (actor) with a value network (critic). The model adds a value head $V_{\phi}(s)$ and use bootstrapped advantages:

$$A_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

3. Research Idea

The thesis primarily focuses on the deterministic TDTSP with a fixed customer set. Therefore, it is important to identify existing gaps early and address them accordingly. We adapt the M1 model to the deterministic TDTSP setting and evaluate three lightweight enhancements:

1) Step-MLP

Instead of using a computationally heavy GRU, we employ a lightweight MLP that takes a step feature input (such as k/N , binary bits, the last three visited nodes, etc) and outputs:

- A small context nudge added to the decoder query
- An optional softmax temperature used to soften the final softmax (dividing logits by the temperature).

2) Cost-Aware Gating (Soft Bias)

Before learning a parametric gating function, we will construct heuristic logits h_i derived from established strategies such as linear travel time, ranking heuristics and two-step lookahead methods like NN+1. These heuristic logits will first be evaluated as standalone policies and then incorporated as fixed log prior biases added to the model's raw attention logits via,

$$a'_i = a_i + \lambda h_i$$

where λ controls the influence of the heuristic. To improve beyond a simple linear bias, we will also explore monotone nonlinear transformations of these heuristic logits. Such as piecewise-linear or exponential functions to flexibly shape the gating toward favouring

shorter or cheaper moves. Only if these fixed heuristic based gating methods consistently improve performance, we will proceed to learning a parametric gating function.

3) Time Slicing with Safe Refresh

Instead of feeding the encoder with the full day’s time series, we feed only a forward window of traffic from the current decision time t . The window can include either all future values or a specific slice $[t, \dots, t + W]$. The decoder continues to receive the current-time travel time vector $w(t)$ as usual. Choosing size of the W will be critical (on validation):

- **Experiment with different W :** Compare the performance on full future with different W (start small and increase it) using the mean tour time, 95th percentile of the tour time and encoder memory.
- **k -moves sanity checks:** For each instance (including the start time), we perform a quick greedy rollout for the next k decisions using the candidate W . If the estimated time of arrival for those k moves stays within $[t, \dots, t + W]$, then W is likely sufficient. Otherwise, we increase W for that start time.

What if the window “expires” ($t + W \leq \text{current_time}$) ?

To address this with three fallback mechanisms:

- Buffer rule: Check whether the next two or three greedy lookahead moves still fall within the selected window.
- One-time fallback: If the current time hits the boundary, refresh the encoder once.
- Periodic refresh: Automatically refresh the encoder at fixed time intervals, regardless of estimated time of arrival.

4. Data Foundation

4.1 Primary Datasets: Real-World City Data

Our experiments use primary datasets from [TDRouting](#), which provides comprehensive, cross-city benchmark instances for the TDTSP under deterministic, non-waiting behaviour (respecting the FIFO property). These datasets cover four major cities. Such as Rome, Paris, London and Bonn. They are fully public, versioned and reproducible.

The Bonn dataset is originally designed for time dependent vehicle routing problems but we will adapt and convert it as needed for the TDTSP setting to maintain compatibility.

4.2 Synthetic Dataset

We generate a synthetic dataset with controlled traffic profiles, including:

- Classic smooth profiles (off-peak, AM-peak, midday, PM-peak), capturing typical daily traffic variations.

- New “chaotic” or “outlier” profiles with abrupt or random fluctuations to test robustness against extreme or adversarial traffic patterns not seen in real data.

Customer locations are randomly sampled inside a unit square, with the depot located at its center. To model asymmetric travel times, we start with Euclidean distances and introduce small directional biases.

Each road segment is assigned one of the traffic profiles, with a phase shift applied to avoid synchronized peak times across all arcs. Travel times $g_{ij}(t)$ between location pairs i and j at departure time t are computed over a discretized time grid. We apply isotonic smoothing to ensure the FIFO property, preventing scenarios where departing later could lead to earlier arrival.

Coordinates are scaled to the $[0,1]^2$ range, and travel times are expressed in minutes. For each instance, we store basic statistics used later to normalize data during model training.

4.3 Instance Structure and Format

Each dataset comprises 10 instances with 10–100 customers. Each instance forms a complete directed graph over customers, featuring departure time dependent travel times for every arc:

- topology.txt (mandatory): Continuous piecewise linear travel time functions per arc (i,j) , sampled every 5 minutes, with X as time of day (seconds) and Y as travel time (seconds).
- tw.txt (optional): Time windows per customer, for TDTSP with time windows (TDTSPTW).
- approximations.txt (optional): Per arc length bounds and typical speed profiles for fast approximate routing.

All arc functions satisfy the FIFO property.

4.4 Data Preprocessing and Binning

Time dependent arc travel times are sampled every 5 minutes. For learning, we deterministically rebin these curves into 12 bins per day (2-hour intervals):

- Bin midpoints: $\{1, 3, 5, \dots, 23\}$ hours are used as representative start times.
- Rebinning methods: Use either bin-midpoint values or within bin averages, fixed across experiments.

5. Experimental Setup and Evaluation Plan

5.1 Training and Validation Schedules

- **Dataset:** We start with the Rome-100 dataset.
- **Instance splits:** For Rome-100, instances 0–5 serve as training, 6–7 for validation and 8–9 for testing. Similar splits apply for other city datasets.
- **Augmentation:** Gradually introduce synthetic chaotic profiles by mixing batches or episodes over subsequent epochs to enhance robustness.
- **Training start times:** Each epoch uses three start times per instance, selected via a **cyclic approach** whereby epoch e uses bins $\{e, e + 4, e + 8\} \bmod 12$.
- **Validation/Test:** Fixed start times (06:00, 10:00, 14:00, 18:00, 22:00) are used per instance for consistent evaluation.

5.2 Evaluation and Baselines and Metrics

Datasets

- Zero shot evaluation is performed on Paris-100 and London-100 instances.
- For broader generalization and external validity, the Bonn dataset is also tested.
- Synthetic “outlier” instances provide robustness assessment against worst case time dependencies.

Baselines

- NN (Nearest Neighbor heuristic)
- NN+1 (Nearest Neighbor plus one-step improvement heuristic)
- Rank-NN (Ranking-based NN heuristic variants)
- M1 Model

Evaluation Metrics

- **Mean Tour Time:** Average total travel time across test instances, measuring overall efficiency.
- **95th Percentile Tour Time:** Captures travel time reliability by reflecting worst-case scenarios in 95% of runs; a critical transportation planning metric.
- **Optimality / Heuristic Gap:** Percentage gap in travel time relative to the best heuristic baseline, quantifying improvements contributed by the learning method.

6. Proposed Timeline

Task	Time
Research literature	June 2025
Decide solution approaches	July 2025
Submit proposal and First Idea Talk	August 2025
Data generator & replicate baseline (M1)	September 2025
Implementation and testing - Step-MLP	September 2025
Implementation & testing - Cost-aware gate	October 2025
Implementation and testing - Time slicing	November 2025
Implementation and Testing - Actor Critic	December 2025
Prepare thesis and deliverables	January 2026
Final Presentation	February 2026

7. Resources

- **Compute & infrastructure** : University GPU cluster
- **Software**: Python, PyTorch, NumPy/SciPy, CUDA/cuDNN, Matplotlib
- **Exploring RL4CO** (if minimal changes needed)
- **Baseline** : M1 implementation

8. References

- [1] Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=ByxBFsRqYm>
- [2] Zhang, Z., Liu, H., Zhou, M., & Wang, J. (2023). Solving dynamic traveling salesman problems with deep reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 34(4), 2119–2132. <https://doi.org/10.1109/TNNLS.2021.3105905>
- [3] Wu, G., Zhang, Z., Liu, H., & Wang, J. (2021). Solving time-dependent traveling salesman problem with time windows with deep reinforcement learning. 2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC) (pp. 558–563). IEEE. <https://ieeexplore.ieee.org/document/9658956>
- [4] Chen, D., Imdahl, C., Lai, D., & Van Woensel, T. (2025). The dynamic traveling salesman problem with time-dependent and stochastic travel times: A deep reinforcement learning approach. *Transportation Research Part C: Emerging Technologies*, 172, 105022. <https://doi.org/10.1016/j.trc.2025.105022>

- [5] Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (NeurIPS 2015) (pp. 2692–2700). <https://papers.nips.cc/paper/5866-pointer-networks.pdf>
- [6] Adamo, T., Ghiani, G., & Guerriero, E. (2024). VRPLib: a library of instances for time-dependent routing problems [Preprint]. Retrieved August 21, 2025, from https://www.tdrouting.com/VRPLib/instances_format.pdf
- [7] Rosenkrantz, D. J., Stearns, R. E., & Lewis II, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3), 563–581. <https://doi.org/10.1137/0206041>
- [8] Dorigo, M., & Gambardella, L. M. (1997). Ant colonies for the travelling salesman problem. *BioSystems*, 43(2), 73–81. [https://doi.org/10.1016/S0303-2647\(97\)01708-5](https://doi.org/10.1016/S0303-2647(97)01708-5)